Keeper Memory

Répartition des tâches — ETHGlobal Open Agents 2026

Marius · Cyriac · Nohem | CNM Agency

Légende : 🔵 **Backend** 🟢 **Frontend** 🟠 **Smart Contracts**

🔵 BACKEND — Python / FastAPI / Pipeline

Setup & Infrastructure

- [x] ~~Initialiser le projet Python (pyproject.toml + dépendances : click, fastapi, slither, cognee, x402-py)~~
- [x] ~~Setup .env complet (ANTHROPIC_API_KEY, KEEPERHUB_API_KEY, OG_PRIVATE_KEY, NAMESPACE_API_KEY…)~~
- [x] ~~Scaffold structure de fichiers (keeper_memory/ avec tous les modules)~~
- [x] ~~CLI Click — commandes : audit, init, --dev, --local, --help~~
- [x] ~~FastAPI server.py — routes /audit et /audit/local~~

x402 — Paiements

- [ ] x402 middleware serveur (server.py — verify_x402_payment())
- [ ] x402 client CLI (payments/x402_client.py — envoie le paiement)
- [x] ~~Calcul du prix dynamique dans Phase 0 (calculate_price() selon nb fichiers)~~
- [ ] Affichage du prix + confirmation avant paiement dans le terminal
- [x] ~~Mode --local : bypass x402, mémoire locale uniquement~~

Pipeline — Phase 0 : Resolve

- [ ] Résoudre adresse ETH onchain → fichiers Solidity (via Etherscan API)
- [x] ~~Résoudre chemin local .sol ou répertoire → liste de fichiers~~
- [x] ~~Détection fork upstream : KNOWN_UPSTREAMS (Uniswap, Aave, OZ…)~~
- [ ] detect_upstream() — réduction scope au diff uniquement si fork détecté
- [x] ~~Retourner ResolvedContract avec liste de fichiers + upstream optionnel~~

Pipeline — Phase 1 : Inventory

- [x] ~~Parse structurel sans LLM — extraction fonctions, modifiers, events~~
- [x] ~~Flags rapides : delegatecall, unchecked, assembly, selfdestruct~~
- [ ] load_known_findings() depuis Cognee pour ce contract_id
- [ ] is_duplicate() — cross-référence par pattern_hash avant chaque finding

Pipeline — Phase 2 : Slither

- [x] ~~Lancer Slither — sortie JSON filtrée~~
- [ ] Sur fork détecté : analyser diff only (fichiers modifiés uniquement)
- [x] ~~Parser output Slither → liste structurée de findings avec severity~~

Pipeline — Phase 3 : Triage

- [ ] Map/Reduce par fichier avec claude-haiku-4-5
- [ ] Calcul risk_score 0–10 par fichier
- [ ] Gate : score < 3 → verdict SAFE, arrêt du pipeline

Pipeline — Phase 4 : Investigation Agent

- [ ] Agent claude-sonnet-4-6, 30 turns max
- [ ] System prompt adversarial : 'Read code adversarially before checklist'
- [ ] Outil 1 : read_contract(file, fn) — lit le source Solidity d'une fonction
- [ ] Outil 2 : search_pattern(regex) — grep sur tous les .sol
- [ ] Outil 3 : get_call_graph(contract) — call graph via Slither
- [ ] Outil 4 : get_storage_layout(contract) — détection collisions storage
- [ ] Outil 5 : query_memory(query, collective) — Cognee local + collectif
- [ ] Outil 6 : simulate_call(sig, args) — simulation Tenderly (optionnel S2)
- [ ] Outil 7 : anchor_finding(pattern_hash, root_hash) — KeeperHub MCP
- [ ] Niveaux de confiance : SUSPECTED → LIKELY → CONFIRMED
- [ ] Anchor déclenché uniquement pour LIKELY et CONFIRMED

Pipeline — Phase 5 : Anchor (filet)

- [ ] Direct Execution API KeeperHub — vérifier anchors de l'agent
- [ ] Compléter les anchors manqués par Phase 4
- [ ] poll_execution() — attendre confirmation tx hash
- [ ] Log warning si anchor échoue, continuer sans bloquer

Pipeline — Phase 6 : Report

- [ ] Générer rapport JSON avec tous les findings
- [ ] Format : severity, file, line, description, prior_audit_ref, fix_sketch, onchain_proof
- [ ] Déclencher mint_cert() ENS si 0 findings HIGH
- [ ] Affichage terminal final : verdict, findings, liens onchain + ENS

Mémoire Cognee

- [ ] cognee_client.py : wrapper Cognee (search, add, local scope)
- [ ] Mémoire locale ~/.keeper-memory/memory/ — privée par machine
- [ ] query_memory() : local uniquement (free) ou local + collectif (paid)
- [ ] Normalisation snippets : yieldAmount → VAR_AMOUNT, feeRecipient → ADDR_RECIPIENT
- [ ] Jamais stocker : contract_address, project_name, raw_code
- [ ] Bootstrap mémoire collective — parser Rekt.news top 100 (~500 patterns)
- [ ] Bootstrap — Immunefi disclosures publics (~300 patterns)
- [ ] Bootstrap — Audit reports publics OZ / Trail of Bits (~1000 patterns)
- [ ] Opt-in contribution : prompt Y/N + calcul crédits x402 (-0.25 USDC/pattern)

0G Storage

- [ ] Installer @0gfoundation/0g-ts-sdk + ethers (Node.js)
- [ ] 0g/0g_upload.js : JSON payload → rootHash (Merkle root via MemData)
- [ ] 0g/0g_download.js : rootHash → JSON (téléchargement depuis réseau 0G)
- [ ] storage/zero_g_client.py : store_pattern() appelle upload.js via subprocess
- [ ] storage/zero_g_client.py : retrieve_pattern() appelle download.js
- [ ] 0G KV Storage : kv_set(STREAM_ID, pattern_hash, json) + kv_get()
- [ ] Tester upload + download sur testnet Galileo

KeeperHub Integration

- [ ] keeper/direct_api.py : anchor_contribution() via Direct Execution API
- [ ] POST /api/execute/contract-call avec pattern_hash + root_hash_0g
- [ ] poll_execution() : GET /api/execute/{id}/status → transactionHash
- [ ] keeper/mcp_tools.py : anchor_finding_mcp() via MCP (Option B démo)
- [ ] Appel MCP : get_wallet_integration → execute_workflow → get_execution_logs

ENS / Namespace SDK

- [ ] ens/subname_manager.py : mint_cert() après chaque audit
- [ ] POST Namespace API — label + parentName + textRecords
- [ ] Text records : verdict, high_count, medium_count, audit_date, tx_proof, report_hash
- [ ] mint_auditor_subname() pour les contributeurs opt-in
- [ ] Tester résolution ENS décentralisée : ns.get_text(subname, 'verdict')

🟢 FRONTEND — Next.js / UX Terminal

Setup Terminé

- [x] ~~Initialiser projet Next.js (TypeScript, Tailwind)~~
- [x] ~~Configurer proxy / connexion à l'API FastAPI backend (NEXT_PUBLIC_API_URL)~~
- [x] ~~Setup variables d'environnement + mode mock (NEXT_PUBLIC_USE_MOCK) Pages & Dashboard Terminé~~
- [x] ~~Page d'accueil — landing + explication du projet + stack + pricing~~
- [x] ~~Page /memory — statistiques mémoire collective + sources + anonymisation~~
- [x] ~~Page /history — liste des audits avec verdict, risk score, findings count~~
- [x] ~~Page /audit/[id] — rapport complet avec findings expandables~~
- [x] ~~Findings avec severity tags (HIGH/MED/LOW) + confidence~~
- [x] ~~Code diff viewer : extrait vulnérable + fix suggéré (fix_sketch)~~
- [x] ~~Section 'Memory hits' : référence aux hacks similaires (Euler, Ronin…)~~
- [x] ~~Export rapport JSON (bouton download)~~
- [x] ~~Affichage onchain proof : lien Etherscan tx KeeperHub~~
- [x] ~~Badge ENS : lien cliquable vers certified.keeper-memory.eth~~
- [x] ~~Wallet banner : adresse + solde USDC + alerte balance faible~~
- [x] ~~Statistiques footer historique : total audits, HIGH findings, USDC spent, certified~~
- [x] ~~Types TypeScript complets (lib/types.ts)~~
- [x] ~~Mock data réaliste basée sur EulerVault.sol déployé (lib/mock-data.ts)~~
- [x] ~~API layer avec fallback mock / real (lib/api.ts)~~
- [x] ~~SSE streamAudit() — infrastructure prête, events mappés~~

À faire — Reste mineur

- [ ] Connecter l'API réelle : passer NEXT_PUBLIC_USE_MOCK=false quand le backend est prêt
- [ ] Dev banner : bandeau orange si mode --dev actif (composant global layout)
- [ ] Message upgrade free tier --local : 'Unlock collective memory → X USDC'
- [ ] Vérifier le responsive mobile pour la démo

🟠 SMART CONTRACTS — Solidity / KeeperHub / ENS

AnchorRegistry.sol

- [x] ~~Écrire AnchorRegistry.sol : event Anchored(patternHash, rootHash0G, contributor, timestamp)~~
- [x] ~~Fonction anchor(bytes32 pHash, bytes32 root0G) external~~
- [x] ~~Events only — gas minimal, pas de storage onchain~~
- [x] ~~Déployer sur eth Sepolia (Hardhat ou Foundry)~~
- [x] ~~Vérifier le contrat sur Etherscan eth Sepolia~~
- [x] ~~Tester anchor() call via script ou Remix~~
- [x] ~~Copier ANCHOR_REGISTRY_ADDRESS dans .env~~

KeeperHub Workflow

- [x] ~~Créer workflow KeeperHub : Trigger Webhook → Write Contract~~
- [x] ~~Configurer action : web3/write-contract, AnchorRegistry.anchor(hash, root), eth Sepolia~~
- [x] ~~Lier au wallet Para (get_wallet_integration → walletId)~~
- [x] ~~Récupérer ANCHOR_WORKFLOW_ID + URL webhook → .env~~
- [x] ~~Tester workflow manuellement : curl payload → KeeperHub → tx Sepolia~~
- [x] ~~Vérifier tx sur Etherscan : event Anchored bien émis~~

Para MPC Wallet

- [x] ~~Récupérer walletId Para depuis KeeperHub (get_wallet_integration MCP)~~
- [x] ~~Financer wallet en ETH Sepolia (sepoliafaucet.com)~~
- [x] ~~Tester write op via Direct Execution API~~
- [x] ~~Vérifier gas estimation automatique (nonce géré par KeeperHub)~~

ENS — Setup Onchain

- [x] ~~Enregistrer keeper-memory.eth sur testnet ENS~~
- [x] ~~Configurer subdomain : certified.keeper-memory.eth~~
- [x] ~~Configurer subdomain : auditors.keeper-memory.eth~~
- [x] ~~Configurer résolver Namespace.ninja (offchain, gasless)~~
- [x] ~~Obtenir NAMESPACE_API_KEY → .env~~
- [x] ~~Tester mint subname via API Namespace~~
- [x] ~~Tester résolution décentralisée : ens.get_text('vault-0x...certified.keeper-memory.eth', 'verdict')~~
- [x] ~~Configuer les text records~~

Vérification indépendante

- [x] ~~Script de vérification chaîne complète : tx KeeperHub → rootHash 0G → pattern JSON~~
- [x] ~~Étape 1 : Etherscan → event Anchored → { patternHash, rootHash0G }~~
- [ ] Étape 2 : 0G StorageScan → rootHash0G → pattern JSON complet
- [ ] Étape 3 : SHA-256(normalized_snippet) == patternHash anchoré → intégrité ✓
- [ ] Tester flux complet : find → store 0G → anchor KeeperHub → résoudre ENS

Keeper Memory — Contracts & Infrastructure

Déployé par Marius — ETHGlobal Open Agents 2026

AnchorRegistry.sol (v2)

Address : 0x4DC06573aa7b214645f649E4b9412Fe5aEd775F8

Network : Ethereum Sepolia (chainId: 11155111)

Etherscan : https://sepolia.etherscan.io/address/0x4DC06573aa7b214645f649E4b9412Fe5aEd775F8

Fonctions disponibles pour le backend : isAnchored(bytes32 pHash) → bool Vérifier si un pattern est déjà ancré. À appeler en Phase 1 pour la déduplication.

getAnchor(bytes32 pHash) → (bytes32 rootHash0G, address contributor, uint256 timestamp) Récupérer les détails d'un anchor existant.

getTotalAnchors() → uint256 Nombre total de patterns ancrés dans le registre.

anchor(bytes32 pHash, bytes32 root0G) Ancrer un nouveau pattern. Appelé via KeeperHub — ne pas appeler directement.

KeeperHub

API Key : kh_PYIgdBK34A6-OwVP4QTRR-kTJ7vLoZX7

Auth header : Authorization: Bearer kh_PYIgdBK34A6-OwVP4QTRR-kTJ7vLoZX7

Workflow ID : h6sugvw04roch3edpfkzn

Webhook URL : https://app.keeperhub.com/api/workflows/h6sugvw04roch3edpfkzn/webhook

Para Wallet : 0x4DB6Bf931e0AC52E6a35601da70aAB3fF26657C4

Endpoint Direct Execution API : POST https://app.keeperhub.com/api/execute/contract-call GET https://app.keeperhub.com/api/execute/{executionId}/status

Body à envoyer : contractAddress : 0x4DC06573aa7b214645f649E4b9412Fe5aEd775F8 network : 11155111 functionName : anchor functionArgs : ["0xPATTERN_HASH", "0xROOT_HASH_0G"]

Réponse : executionId → à poller sur /status transactionHash → disponible quand status = "completed"

ENS

keeper-memory.eth — owner : 0xe97F62b7Bf214303419189ECD3D6688FdfF30Fd8

certified.keeper-memory.eth — node : 0x5b716a269ff3ff851b1a8e51772139f420b6d11d51f411a1a598e4b63848689d

auditors.keeper-memory.eth — node : 0xe159af539900fa2f5ef430e20adcbcdc0af5817bd8bd771bca3c6761bf475883

ENS Registry Sepolia : 0x00000000000C2E074eC69A0dFb2997BA6C7d2e1e

Public Resolver Sepolia : 0x8FADE66B79cC9f707aB26799354482EB93a5B7dD

Network : Ethereum Sepolia

Mint certificat depuis Python — appel subprocess : subprocess.run(["npx", "ts-node", "scripts/ensManager.ts", "mintCert", contract_address, verdict, str(high_count), str(medium_count), tx_proof, report_hash, audit_date], cwd="./contracts")

Retourne dans stdout : ENS_SUBNAME=contract-XXXXXX.certified.keeper-memory.eth

Mint auditeur depuis Python — appel subprocess :subprocess.run(["npx", "ts-node", "scripts/ensManager.ts", "mintAuditor", wallet_address, str(contributions), str(reputation_score), str(confirmed_findings)], cwd="./contracts")

Retourne dans stdout : ENS_SUBNAME=XXXXXXXX.auditors.keeper-memory.eth

Text records mintés pour un certificat : verdict → "CERTIFIED" ou "FINDINGS_FOUND" high_count → nombre de findings HIGH medium_count → nombre de findings MEDIUM audit_date → "2026-04-26" tx_proof → tx hash KeeperHub report_hash → rootHash 0G

Vérification décentralisée sans backend : Resolver.text(node, "verdict") → "CERTIFIED" | "FINDINGS_FOUND" | ""

EulerVault.sol (contrat vulnérable — démo)

Address : 0x49Ca165Bd6AEe88825f59c557bC52A685e0594B5 **Network** : Ethereum Sepolia **Etherscan** : https://sepolia.etherscan.io/address/0x49Ca165Bd6AEe88825f59c557bC52A685e0594B5 **Vulnérabilité** : Reentrancy dans withdraw() — external call before state update **Rôle** : Contrat cible pour la démo — l'agent doit détecter la reentrancy, l'ancrer via KeeperHub, et minter le certificat ENS.

.env complet backend

ANTHROPIC_API_KEY=

KEEPERHUB_API_KEY=kh_PYIgdBK34A6-OwVP4QTRR-kTJ7vLoZX7

ANCHOR_REGISTRY_ADDRESS=0x4DC06573aa7b214645f649E4b9412Fe5aEd775F8

ANCHOR_WORKFLOW_ID=h6sugvw04roch3edpfkzn

KEEPERHUB_WEBHOOK_URL=https://app.keeperhub.com/api/workflows/h6sugvw04roch3edpfkzn/webhook

ENS_NODE_CERTIFIED=0x5b716a269ff3ff851b1a8e51772139f420b6d11d51f411a1a598e4b63848689d

ENS_NODE_AUDITORS=0xe159af539900fa2f5ef430e20adcbcdc0af5817bd8bd771bca3c6761bf475883

ENS_PARENT_CERT=certified.keeper-memory.eth

ENS_PARENT_AUDITORS=auditors.keeper-memory.eth

OG_PRIVATE_KEY=

OG_RPC_URL=https://evmrpc-testnet.0g.ai

OG_INDEXER_RPC=https://indexer-storage-testnet-turbo.0g.ai

ALCHEMY_API_KEY= demandez moi

ETHERSCAN_API_KEY= demandez moi

Scripts disponibles (contracts/)

scripts/deploy.ts → Déploie AnchorRegistry scripts/deployEulerVault.ts → Déploie EulerVault (démo) scripts/verify.ts → Vérifie un contrat sur Etherscan scripts/setupENS.ts → Crée certified + auditors subdomains scripts/ensManager.ts → Mint cert ou auditor subname scripts/verifyChain.ts → Vérifie la chaîne complète d'un pattern